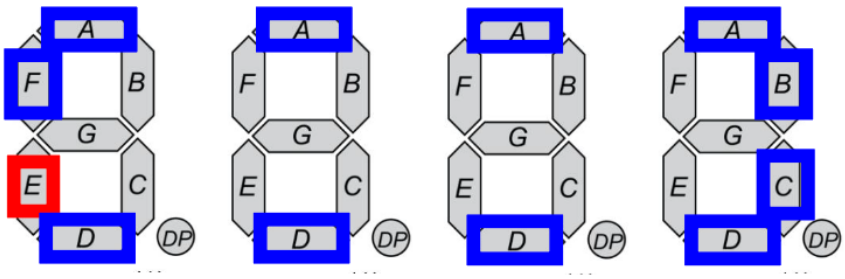
4번 문제 설명 및 코드 설명

- **문제 4**

4-1. P1.1 스위치를 빠르게 두 번(더블 클릭, 두 간격 0.3s 이내일 때) 누르면 세그먼트의 테두리 중 한 칸이 점등

4-2. 엔코더를 시계방향으로 돌리면 세그먼트의 점등 위치가 테두리를 따라 시계방향으로 이동, 반시계 방향으로 돌리면 세그먼트의 점등도 테두리를 따라 반시계방향으로 이동 **(이동경로 파란색으로 표시)**

4-3. 세그먼트의 점등 위치를 가장 왼쪽 아래에 위치 시키면 키패드 활성화 및 숫자 입력시 백, 십, 일의 자리에 해당 3자리 숫자 출력 **(요구 위치 빨간 색으로 표시)**



4-3. 숫자 입력 중 #버튼 누르면 처음부터 입력

4-4. 이때 비밀번호 486입력 후 P1.1 스위치 push하면 세그먼트에 'LOVE' 0.5초마다 토글
(추가점수5)
LOVE 메시지 출력을 화려하게 꾸며보십시오.

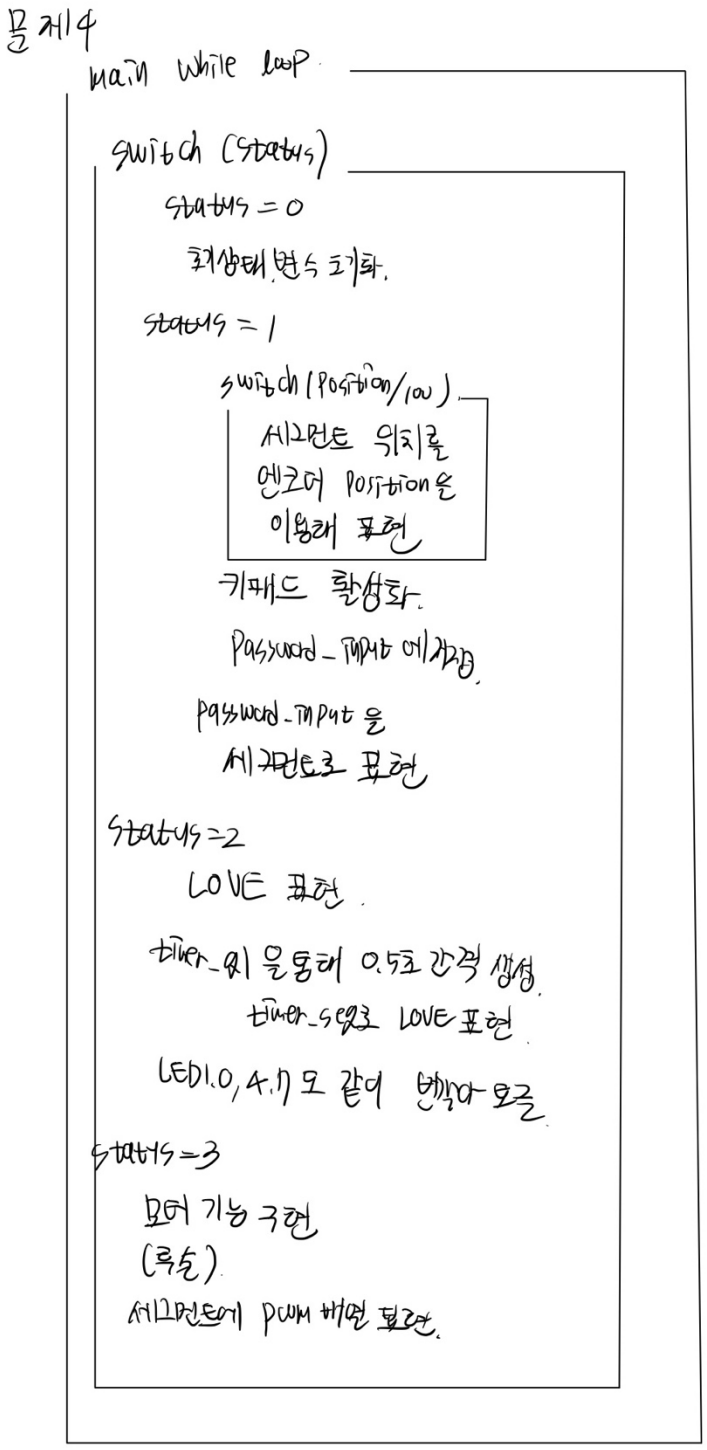
4-5. P1.1 스위치 누르면 모터가 10바퀴마다 방향을 달리함과 동시에 486의 shift된 파워로 회전 및 현재 동작중인 PWM값 세그먼트 출력
(ex / 첫 10바퀴 : 486/1000의 PWM값으로 시계방향 회전
> 두 번째 10바퀴 : 864/1000의 PWM값으로 반시계방향 회전
> 세 번째 10바퀴 : 648/1000의 PWM값으로 시계방향 회전
> 네 번째 10바퀴 : 486/1000의 PWM값으로 반시계방향 회전
> …)

4-6. P1.1 스위치 누르면 초기 상태로 복귀

코드 설명

- 코드 구조

-


문제4
main while loop
switch (status)
status = 0
초기상태 변수 초기화.
status = 1
switch (position/low)
세그먼트 위치를
엔코더 position을
이용해 표현
키패드 활성화.
password_input 에저장.
password_input 을
세그먼트로 표현
status = 2
LOVE 표현.
timer_ms 을통해 0.5초 간격 생성.
timer_step로 LOVE표현.
LED1.0, 4.1 모 같이 번갈아 보글
status = 3
모터 기능 구현
(속도)
세그먼트에 pwm 비율 표현.

-

Port 1 Interrupt

더블클릭 구현

엔코더 인터럽트

status≠3이면

position 값 제어

status=3 이면 (모터 가동)

바퀴수 카운트

Timer A0 Interrupt

timer_ms

LOVE 깜미기구현

timer_sec 제어

변수 설명

```
int position = 0;
```
- 세그먼트에 표시되어야 할 position 변수.

```
unsigned int love[4] = {0x83, 0xDB, 0xD3, 0X8F}; // L O V E
```
- LOVE 에 해당하는 bit를 배열로 선언

```
unsigned int number[10] = { 0xDB, 0x50, 0x1F, 0x5D, 0xD4,
0xCD, 0xCF, 0xD8, 0xDF, 0xDD };
```
- 0~9 까지 숫자 세그먼트 표현을 배열로 선언

```
unsigned int button_pressed = 0;
```
- 앞선 3번 문제와 동일한 더블클릭 구현 변수

```
unsigned int timer_gl = 0;
```
- 글로벌 타이머

```
unsigned int timer_seg = 0;
```
- 세그먼트 타이머

```
volatile unsigned int status = 0;
```
- 현재 상태(state)를 제어하는 변수

```
unsigned short keypad_enable = 0;
```
- 키패드의 활성화 여부를 결정하는 변수

```
int password_input[3] = {10, 10, 10}; // to prevent unwanted
action
```
- 3번문제와 동일

```
int password[3] = {4, 8, 6};
```
- 주어진 비밀번호 (4 8 6)

```
unsigned short password_count = 0;
```
- 키패드 입력 자릿수

```
unsigned short i = 0;
```
- 패스워드가 맞는지 확인하는 함수의 for 문에서 사용할 변수

```
unsigned int cycle_counter = 0;
```
- 엔코더의 카운터

```
int cycle = 9; // 10 cycles including 0
```
- 10바퀴를 세기 위한 바퀴 수 (0을 포함하여 9로 설정)

```
volatile unsigned int pwm[3] = {4, 8, 6};
```
- 4 8 6 pwm을 배열로 선언

```
unsigned short motor_flag = 0;
```
- 모터 회전 방향을 결정하는 motor flag 변수

문제를 해결한 주요 logic

1. 더블클릭 Logic

1.1) 앞서 설명한 문제 3 의 더블클릭 logic 과 완벽히 동일하다.

2. 키패드 logic

2.1) 문제 3 과 유사하지만 조금 다르다. 어떤 숫자든 누를 때 마다 현재 자릿수에 저장하고 자릿수 count 를 1 증가시킨다. 즉, 숫자 하나를 누르면 다른 숫자를 누를 필요 없이 바로 다음 자리의 숫자를 입력받을 상태가 된다.

2.2) Password_input 또한 배열의 크기가 3 이 되었을 뿐 문제 3 과 동일한 Logic 으로 동작한다.

3. 패스워드가 맞는지 확인하는 logic

3.1) 앞선 문제 3 과 동일하다.

4. LOVE 표현

4.1) 0.5 초마다 LOVE 를 켜고, 다른 0.5 초마다 LED 를 껐다 켠다.

4.2) 1 초마다 LOVE 배열의 순서를 한 칸씩 옮겨 LOVE 글자가 변하게 하였다.

5. 10 바퀴 회전 count logic

5.1) 앞선 1 번 문제의 logic 과 유사하다.

5.2) 엔코더 한 바퀴를 count 하면 670count 이고, 모터의 회전 방향이 지속적으로 toggle 되므로 어떤 방향으로 회전하던 count 는 증가하도록 설계하였다.

5.3) 엔코더값이 670 보다 커지게 되면 1 바퀴를 돌았다는 뜻이므로 cycle 을 1 감소시키고, 엔코더 카운터를 0 으로 초기화시킨다.

5.4) Cycle 변수를 0 부터 9 까지 count 하고, cycle<0 이 되면 10 바퀴를 회전했다는 의미이므로 다음 동작을 수행한다.

6. 모터 동작 logic

6.1) 본 문제에서 모터의 회전방향은 두개이다. 시계방향과 반시계방향이므로, motor_flag 변수를 선언하여 motor_flag = 0 이면 시계방향, 1 이면 반시계방향으로 회전하게 설계하였다.

6.2) 회전 속도, 즉 Pwm duty 에 사용할 요소들은 pwm 배열로 선언하였다. Pwm[0]에 100, pwm[1]에 10, pwm[2] 에 1 을 곱한 후 더해 pwm 배열을 세자리 숫자로 만들었다. 즉, pwm[3] = {4, 8, 6} 이면 모터는 486/1000 의 duty 로 동작한다.

6.3) Cycle 변수를 이용하여 10 바퀴 회전을 감지하고 나면 motor_flag 를 토글시켜 모터의 회전 방향을 바꾼다.

6.4) 또한 간단한 c 언어 함수를 이용해 pwm 배열의 순서를 오른쪽으로 한칸 shift 한다. 그렇게 되면 {4, 8, 6} 에서 {8, 6, 4}, {6, 4, 8}의 순서로 계속해서 순환하게 된다.

6.5) Motor_flag 와 pwm 을 배열로 선언한 뒤 배열의 순서를 바꾸는 방법을 이용하여 코드를 최대한 간결하게 설계할 수 있었다.

주요 logic 들에 대한 pseudo code

1. 더블클릭

if(버튼이 눌렸을 때){

if(버튼이 1번 눌림 && timer < 0.3초)

더블클릭 발생

버튼이 1번 눌림 = 1;

timer 초기화

Timer interrupt 내부

if(timer > 0.3초)

버튼이 1번 눌림 = 0

2. 모터 정지

과거 엔코더의 위치 = 직전 timer interrupt 발생 시 엔코더의 위치

if(현재 엔코더의 위치 == 과거 엔코더의 위치)

모터 정지

다음 timer interrupt 시 비교를 위해 현재 엔코더의 위치 값 저장

*오차를 줄이려면 ((현재 엔코더의 위치 – 과거 엔코더의 위치) = 엔코더변화량 < 오차) -> 문제에서 사용한 방법.

3. 회전 바퀴 수 계산

if(엔코더 수치 >= 67)

엔코더 수치 = 0

cycle -1

4. 회전 바퀴 수 근사치 계산

-엔코더 인터럽트 발생-

(signed long)원하는 바퀴 수 * 1,000,000 – 1429 수행

if((signed long)원하는 바퀴 수 * 1000 < 0)

모터 정지 (문제 1-4)

5. 문제 4-5

if(motor_flag == 1)

시계방향 회전

pwm = pwm[3] * 100 + pwm[2] *10 + pwm[0]

if(motor_flag == 0)

반시계방향 회전

pwm = pwm[3] * 100 + pwm[2] *10 + pwm[0]

---

if(10바퀴 감지)

motor_flag toggle

pwm 배열 오른쪽으로 한 칸 shift